МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного бюджетного образовательного учреждения высшего образования «РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №1

по дисциплине: Поддержка и тестирование программных модулей

Руководитель: Д.Б. Берестов /И.О. Фамилия/

“__24__”_февраля_2026г.

Исполнитель: Н.А. Оносова /И.О. Фамилия/

“__24__”февраля__2026г.

Пермь, 2026г.

# Оглавление

Задание....3
Структура проекта....3
Создание проекта в Visual Studio....5
Шаг 1. Открытие Visual Studio....5
Шаг 2. Добавление библиотеке классов....6
Шаг 3. Добавление модульных тестов....6
Шаг 4: Построение решения....9

Целью практической работы является освоение навыков модульного тестирования (Unit-тестирования) на платформе .NET с использованием фреймворка MSTest. В ходе работы выполняется создание тестового проекта, написание и выполнение тестов для проверки функциональности класса BankAccount, а также анализ результатов тестирования.

## Задание

Разработать модульные тесты для проверки методов класса BankAccount. Тесты должны проверять:

1. Корректность выполнения операции списания средств (Debit) при валидных значениях;
2. Поведение метода при попытке списания отрицательной суммы;
3. Поведение метода при попытке списания суммы, превышающей текущий баланс.

## Структура проекта

Bank – основной проект, содержащий тестируемый класс BankAccount.

BankTests – проект модульных тестов, содержащий тесты для проверки функциональности класса BankAccount.

Данная структура позволяет разделить логику приложения и тесты, что соответствует принципам модульного тестирования.

Библиотека классов — это отдельный проект, в котором находится основная логика. Она не содержит интерфейса, только код. Так же, отделяет

интерфейс от логики, с ней легко тестировать (через Unit-тесты), упрощает повторное использование кода (например, та же библиотека может использоваться в другом проекте).

WPF-приложение – это графическая оболочка (окна, кнопки, текстовые поля), в которой пользователь работает с приложением. Пользователь видит понятный интерфейс. WPF использует разметку XAML, позволяющую удобно проектировать внешний вид.

Модульные тесты — это проект, в котором автоматически проверяется, что функции работают правильно.

# Создание проекта в Visual Studio

## Шаг 1. Открытие Visual Studio

Открыть Visual Studio, где виднеется главный экран. Выбрать «Создание проекта» (рис.1). Дальше выбрать WPF-приложение, назвав его Bank.

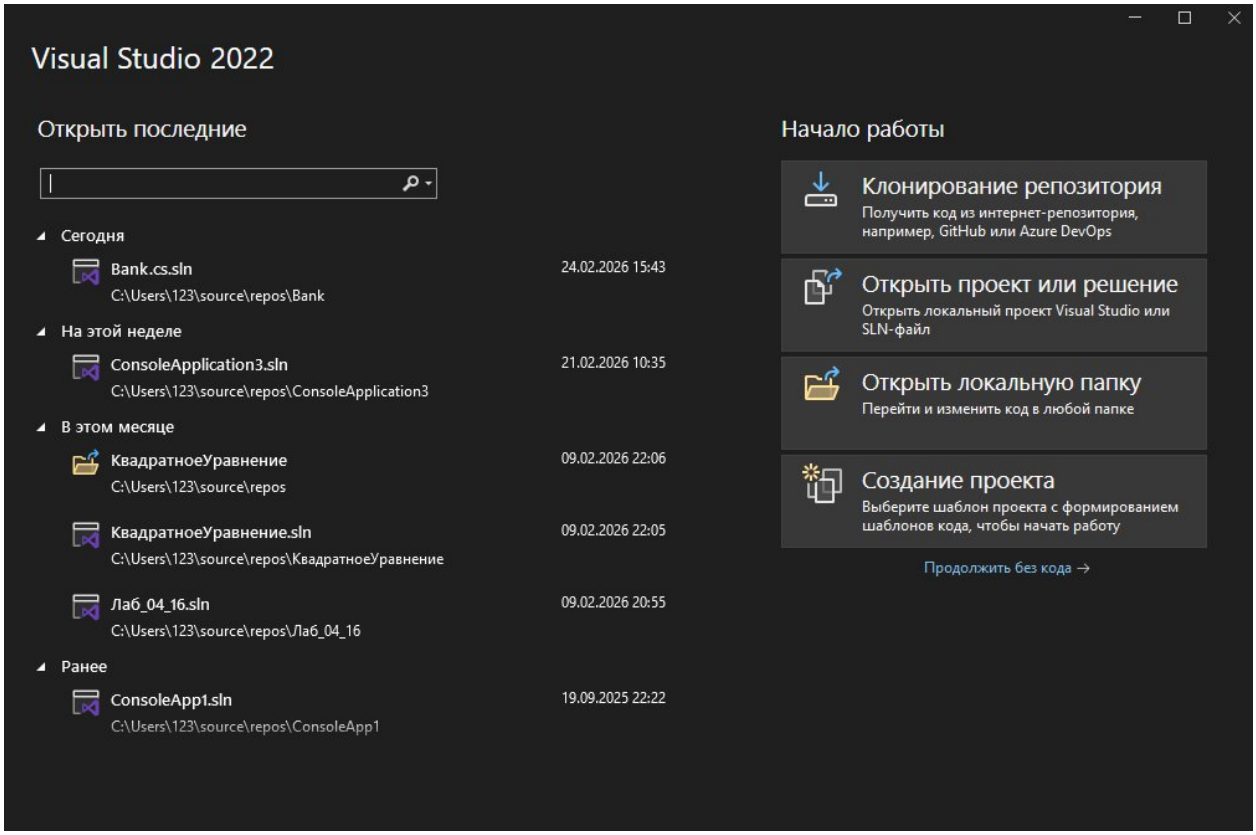



Рис. 1

После создания основного WPF-приложения Bank, следующим шагом является добавление вспомогательных проектов в решение: библиотеки классов и проекта модульных тестов (рис. 2)

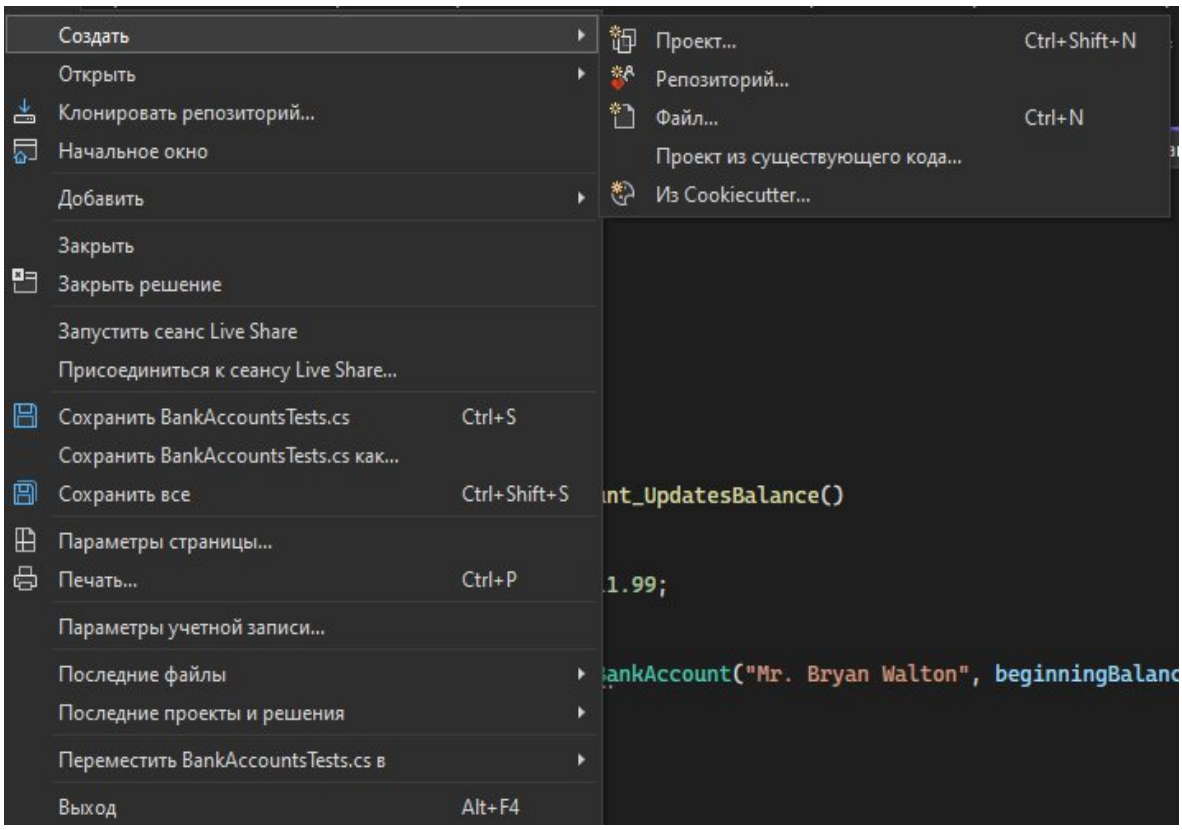



Рис. 2

Шаг 2. Добавление библиотеке классов

В «Обозревателе решений» добавить новый проект, выбрав Библиотеку классов(.NET Framework). Привязать проект к основному.

Шаг 3. Добавление модульных тестов

Создать новый проект модульного тестирования MSTest(.NET Framework)и провести отладку. Так же привязать проект к основному. При первой попытке запуска тестов через обозреватель тестов один из них завершился неудачей. Анализ кода теста Debit_WithValidAmount_UpdatesBalance показал, что ожидаемый результат рассчитывался неверно из-за арифметической ошибки (использование сложения вместо вычитания). После исправления знака «+» на «-» в строке расчета ожидаемого баланса и повторного запуска, тест был успешно пройден. Два других теста, проверяющих исключения, также завершились корректно с первого раза.

Собрать по методичке код для проектов.

## Код для BankAccountsTests.cs:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using BankAccountNS;

namespace BankTests
{
   [TestClass]
   public class BankAccountTests
   {
      [TestMethod]
      public void Debit_WithValidAmount_UpdatesBalance()
      {
         // Arrange
         double beginningBalance = 11.99;
         double debitAmount = 4.55;
         double expected = 7.44;
         BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

         // Act
         account.Debit(debitAmount);

         // Assert
         double actual = account.Balance;
         Assert.AreEqual(expected, actual, 0.001, "Account not debited");
      }
      [TestMethod]
      public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
      {
         // Arrange
         double beginningBalance = 11.99;
         double debitAmount = -100.00;
         BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

         // Act
         try
         {
            account.Debit(debitAmount);
         }
         catch (System.ArgumentOutOfRangeException e)
         {
            // Assert
            StringAssert.Contains(e.Message, BankAccount.DebitAmountLessThanZeroMessage);
            return;
         }

         Assert.Fail("The expected exception was not thrown.");
      }
      [TestMethod]
      public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange()
      {
         // Arrange
         double beginningBalance = 11.99;
         double debitAmount = 20.00;
         BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

         // Act
         try
         {
            account.Debit(debitAmount);
         }
         catch (System.ArgumentOutOfRangeException e)
         {
            // Assert
            StringAssert.Contains(e.Message, BankAccount.DebitAmountExceedsBalanceMessage);
```

```
            return;
        }

        Assert.Fail("The expected exception was not thrown.");
    }
}
      }
```

Код для BankAccount.cs:

```
using System;

namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    public class BankAccount
    {
        private readonly string m_customerName;
        private double m_balance;

        public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
        public const string DebitAmountLessThanZeroMessage = "Debit amount is less than zero";

        private BankAccount() { }

        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }

        public string CustomerName
        {
            get { return m_customerName; }
        }

        public double Balance
        {
            get { return m_balance; }
        }

        public void Debit(double amount)
        {
            if (amount > m_balance)
            {
                throw new ArgumentOutOfRangeException("amount", amount,
DebitAmountExceedsBalanceMessage);
            }

            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount", amount,
DebitAmountLessThanZeroMessage);
            }

            m_balance -= amount;
        }

        public void Credit(double amount)
        {
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
```

```
        }

        m_balance += amount;
    }

    public static void Main()
    {
        BankAccount ba = new BankAccount("Mr. Bryan Walton", 11.99);

        ba.Credit(5.77);
        ba.Debit(11.22);
        Console.WriteLine("Current balance is ${0}", ba.Balance);
    }
}
      }
```

Код для Bank.csproj:

```
<Project Sdk="Microsoft.NET.Sdk">

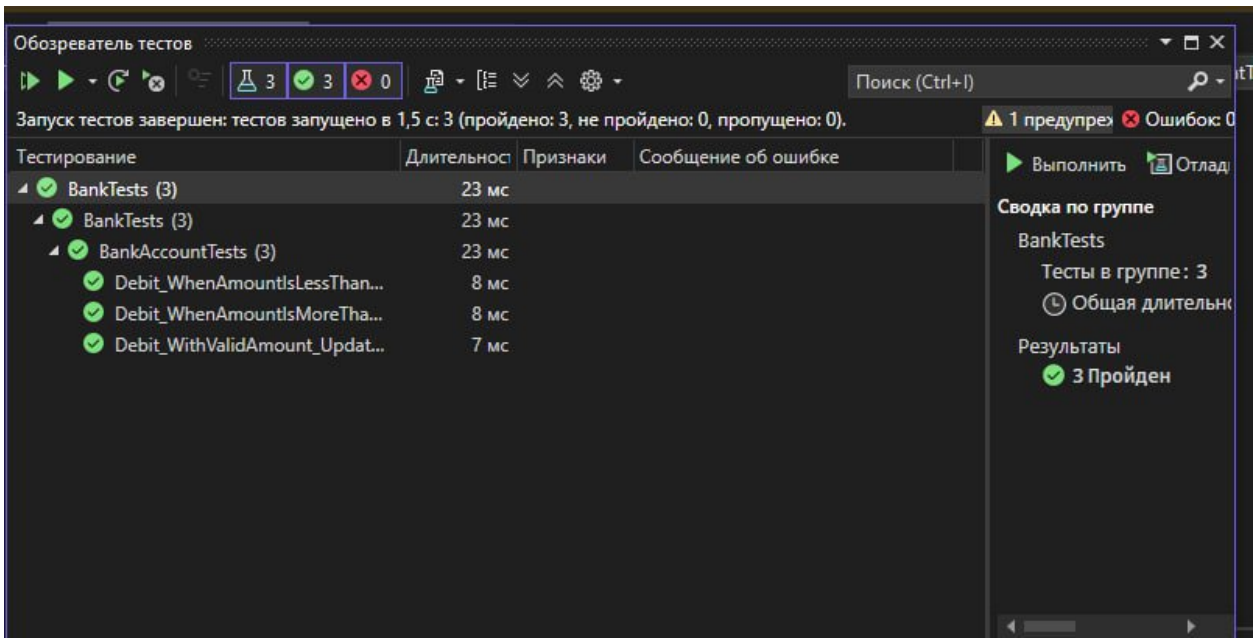
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Теперь, когда код реализован, интерфейс и тесты, необходимо выполнить сборку проекта для его компиляции и последующего запуска.

Шаг 4: Построение решения

В Visual Studio открыть «Обозреватель решений», в верхнем меню выбрать Сборка → Пересобрать решение. Visual Studio скомпилирует все проекты. Проведение тестирования так же (рис. 3). После внесения исправлений все три теста были запущены повторно. Все они успешно прошли проверку, что подтвердило корректность работы метода Debit во всех трех проверяемых сценариях. Код класса BankAccount был дополнен константами для сообщений об ошибках и логикой проверок, соответствующей тестам.




Вывод по работе

В ходе выполнения данной практической работы мною был успешно освоен процесс создания и запуска модульных тестов в среде Visual Studio с применением MSTest. На практическом примере тестирования метода Debit класса BankAccount были реализованы и проверены основные сценарии его работы, включая как позитивные (корректное списание), так и негативные (обработка ошибочных ситуаций). Особенно важным этапом стала отладка, в ходе которой с помощью первого проваленного теста была выявлена и исправлена ошибка в расчетах, что наглядно демонстрирует эффективность методологии TDD (Test-Driven Development) или, как минимум, пользу от написания тестов для контроля качества кода. В итоге, после всех исправлений, все три теста были успешно пройдены, что подтвердило корректность реализации метода и его готовность к использованию.